

梯度下降

尽管梯度下降（[gradient descent](#)⁺）很少直接用于深度学习， 但了解它是理解下一节随机梯度下降算法的关键。例如，由于学习率过大，优化问题可能会发散，这种现象早已在梯度下降中出现。同样地，[预处理](#)（preconditioning）是梯度下降中的一种常用技术， 还被沿用到更高级的算法中。让我们从简单的一维梯度下降开始。

一维梯度下降

为什么[梯度下降算法](#)⁺可以优化目标函数？ 一维中的梯度下降给我们很好的启发。考虑一类连续可微实值函数 $f : \mathbb{R} \rightarrow \mathbb{R}$, 利用泰勒展开，我们可以得到

$$f(x + \epsilon) = f(x) + \epsilon f'(x) + \mathcal{O}(\epsilon^2). \tag{11.3.1}$$

即在一阶近似中， $f(x + \epsilon)$ 可通过 x 处的函数值 $f(x)$ 和一阶导数 $f'(x)$ 得出。我们可以假设在负梯度方向上移动的 ϵ 会减少 f 。为了简单起见，我们选择固定步长 $\eta > 0$, 然后取 $\epsilon = -\eta f'(x)$ 。将其代入泰勒展开式我们可以得到

$$f(x - \eta f'(x)) = f(x) - \eta f'^2(x) + \mathcal{O}(\eta^2 f'^2(x)). \tag{11.3.1}$$

如果其导数 $f'(x) \neq 0$ 没有消失，我们就能继续展开，这是因为 $\eta f'^2(x) > 0$ 。此外，我们总是可以令 η 小到足以使高阶项变得不相关。因此，

$$f(x - \eta f'(x)) \lesssim f(x). \tag{11.3.3}$$

这意味着，如果我们使用

$$x \leftarrow x - \eta f'(x) \tag{11.3.4}$$

来迭代 x , 函数 $f(x)$ 的值可能会下降。因此，在梯度下降中，我们首先选择初始值 x 和常数 $\eta > 0$, 然后使用它们连续迭代 x , 直到停止条件达成。例如，当梯度 $|f'(x)|$ 的幅度足够小或迭代次数达到某个值时。

下面我们来展示如何实现梯度下降。为了简单起见，我们选用目标函数 $f(x) = x^2$ 。尽管我们知道 $x = 0$ 时 $f(x)$ 能取得最小值， 但我们仍然使用这个简单的函数来观察 x 的变化。

```
%matplotlib inline
import numpy as np
import torch
from d2l import torch as d2l
def f(x): # 目标函数
    return x ** 2

def f_grad(x): # 目标函数的梯度(导数)
    return 2 * x
```

接下来, 我们使用 $x = 10$ 作为初始值, 并假设 $\eta = 0.2$ 。使用梯度下降法迭代 x 共10次, 我们可以看到, x 的值最终将接近最优解。

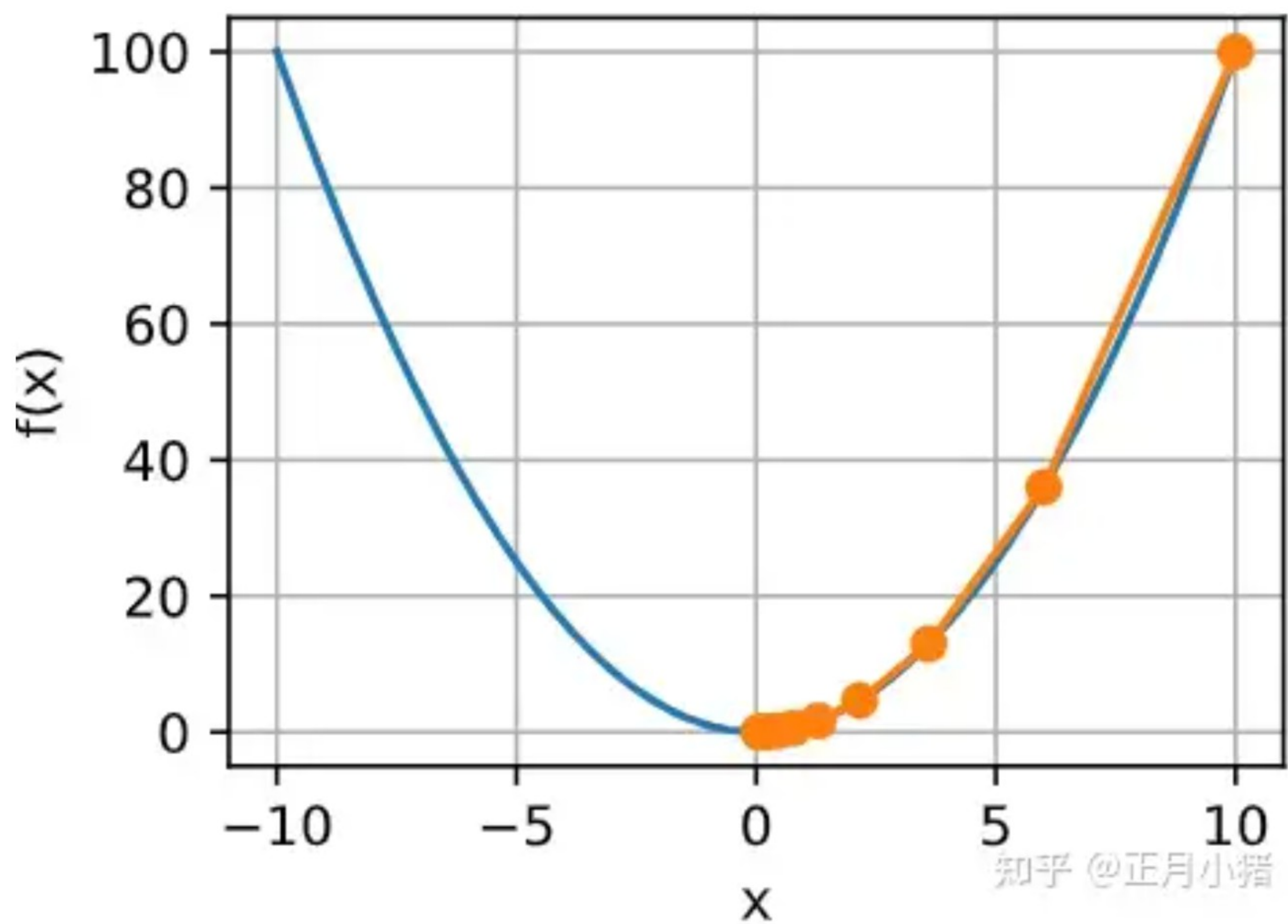
```
def gd(eta, f_grad):
    x = 10.0
    results = [x]
    for i in range(10):
        x -= eta * f_grad(x)
        results.append(float(x))
    print(f'epoch 10, x: {x:f}')
    return results

results = gd(0.2, f_grad)
epoch 10, x: 0.060466
```

对进行 x 优化的过程可以绘制如下。

```
def show_trace(results, f):
    n = max(abs(min(results)), abs(max(results)))
    f_line = torch.arange(-n, n, 0.01)
    d2l.set_figsize()
    d2l.plot([f_line, results], [[f(x) for x in f_line], [
        f(x) for x in results]], 'x', 'f(x)', fmts=['-', '-o'])

show_trace(results, f)
```

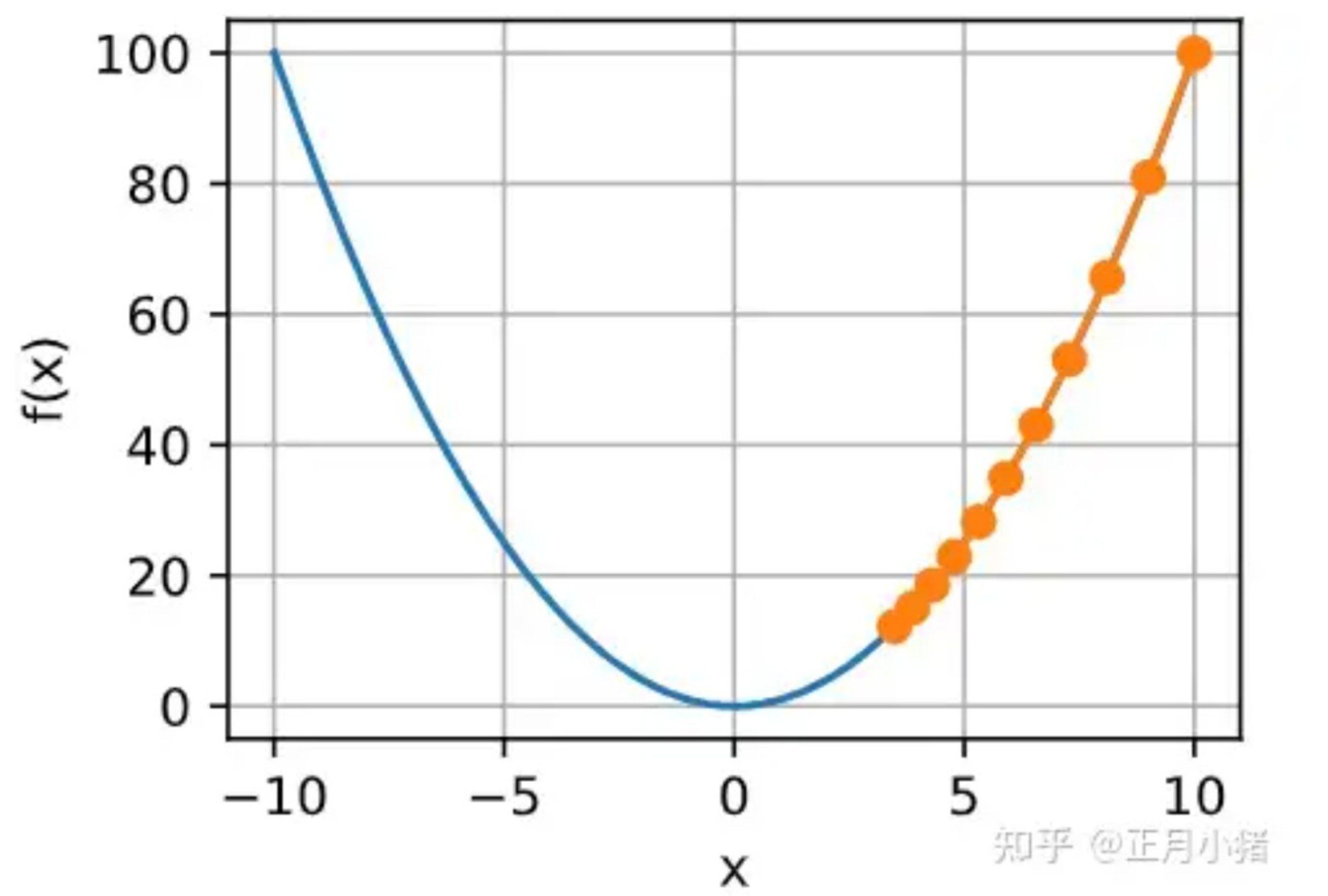



学习率

学习率（learning rate）决定目标函数能否收敛到局部最小值，以及何时收敛到最小值。学习率 η 可由算法设计者设置。请注意，如果我们使用的学习率太小，将导致 x 的更新非常缓慢，需要更多的迭代。例如，考虑同一优化问题中 $\eta = 0.05$ 的进度。如下所示，尽管经过了10个步骤，我们仍然离最优解很远。

```
show_trace(gd(0.05, f_grad), f)
```

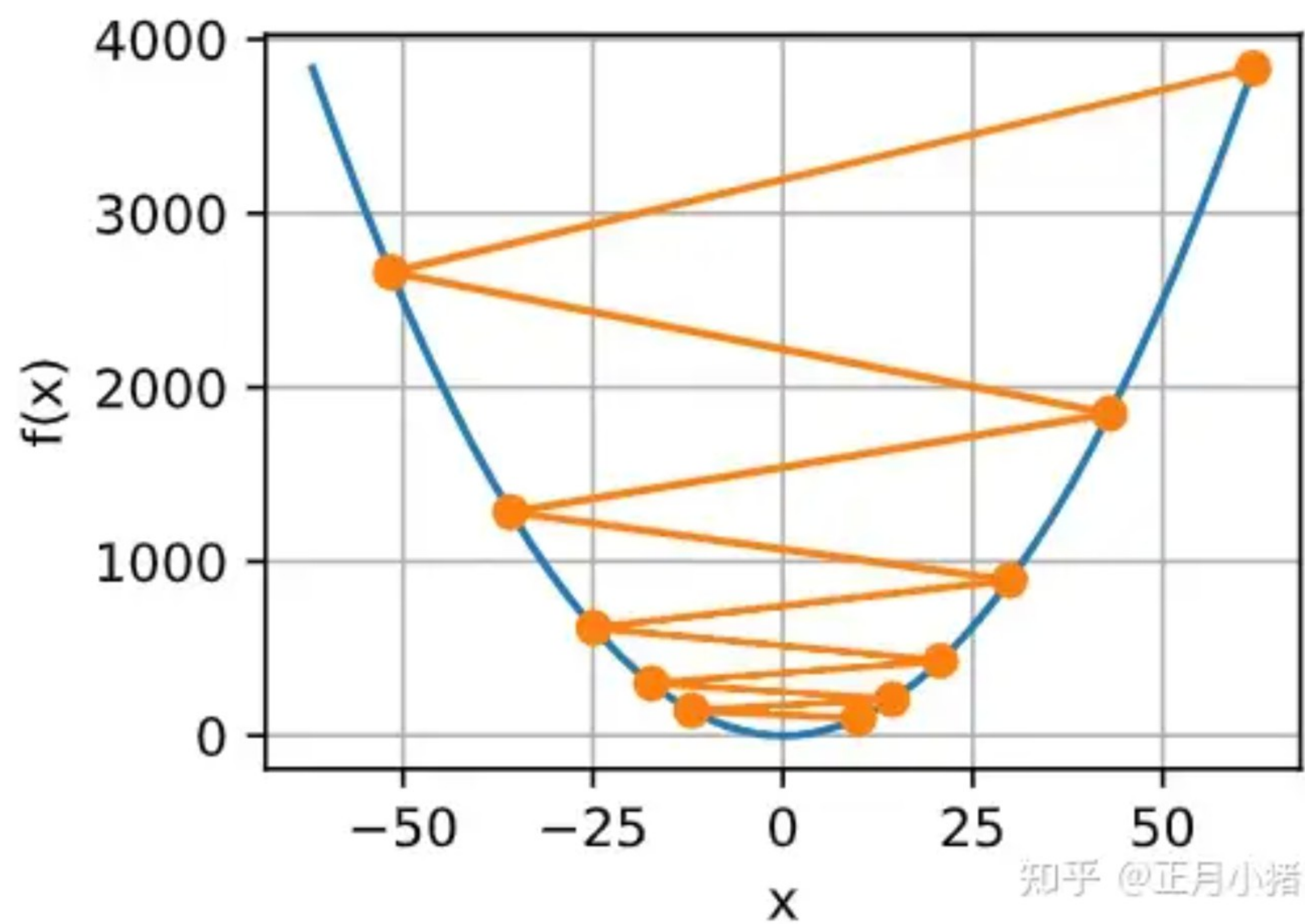
输出结果：
epoch 10, x: 3.486784



相反，如果我们使用过高的学习率， $|\eta f'(x)|$ 对于一阶泰勒展开式可能太大。也就是说，(11.3.1) 中的 $\mathcal{O}(\eta^2 f''(x))$ 可能变得显著了。在这种情况下， x 的迭代不能保证降低 $f(x)$ 的值。例如，当学习率为 $\eta = 1.1$ 时， x 超出了最优解 $x = 0$ 并逐渐发散。

```
show_trace(gd(1.1, f_grad), f)
```

输出结果：
epoch 10, x: 61.917364



局部最小值

为了演示非凸函数的梯度下降，考虑函数 $f(x) = x \cdot \cos(cx)$ ，其中 c 为某常数。这个函数有无穷多个局部最小值。根据我们选择的学习率，我们最终可能只会得到许多解的一个。下面的例子说明了（不切实际的）高学习率如何导致较差的局部最小值。

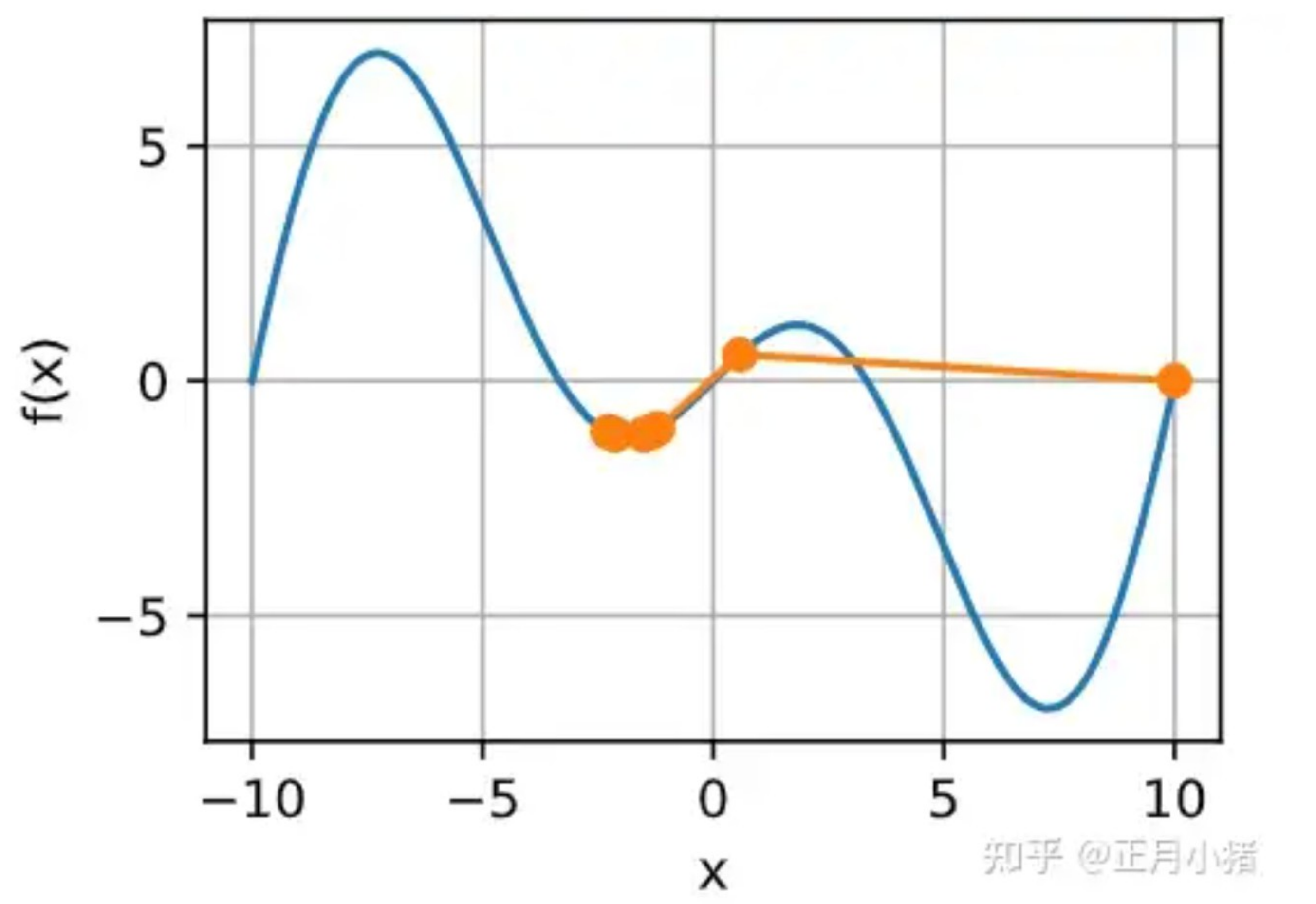
```
c = torch.tensor(0.15 * np.pi)

def f(x): # 目标函数
    return x * torch.cos(c * x)

def f_grad(x): # 目标函数的梯度
    return torch.cos(c * x) - c * x * torch.sin(c * x)

show_trace(gd(2, f_grad), f)
```

输出结果：
epoch 10, x: -1.528166



多元梯度下降

现在我们对单变量的情况有了更好的理解，让我们考虑一下 $\mathbf{x} = [x_1, x_2, \dots, x_d]^\top$ 的情况。即目标函数 $f: \mathbb{R}^d \rightarrow \mathbb{R}$ 将向量映射成标量。相应地，它的梯度也是多元的，它是一个由 d 个偏导数组成的向量：

$$\nabla f(\mathbf{x}) = \left[\frac{\partial f(\mathbf{x})}{\partial x_1}, \frac{\partial f(\mathbf{x})}{\partial x_2}, \dots, \frac{\partial f(\mathbf{x})}{\partial x_d} \right]^\top. \tag{11.3.5}$$

梯度中的每个偏导数元素 $\partial f(\mathbf{x}) / \partial x_i$ 代表了当输入 x_i 时 f 在 $\mathbf{x}$ 处的变化率。和先前单变量的情况一样，我们可以对多变量函数使用相应的泰勒近似来思考。具体来说，

$$f(\mathbf{x} + \boldsymbol{\epsilon}) = f(\mathbf{x}) + \boldsymbol{\epsilon}^\top \nabla f(\mathbf{x}) + \mathcal{O}(\|\boldsymbol{\epsilon}\|^2). \tag{gd-multi-taylor 11.3.6}$$

换句话说，在 $\boldsymbol{\epsilon}$ 的二阶项中，最陡下降的方向由负梯度 $-\nabla f(\mathbf{x})$ 得出。选择合适的学习率 $\eta > 0$ 来生成典型的梯度下降算法：

$$\mathbf{x} \leftarrow \mathbf{x} - \eta \nabla f(\mathbf{x}). \tag{11.3.7}$$

这个算法在实践中的表现如何呢? 我们构造一个目标函数 $f(\mathbf{x}) = x_1^2 + 2x_2^2$, 并有二维向量 $\mathbf{x} = [x_1, x_2]^\top$ 作为输入, 标量作为输出。梯度由 $\nabla f(\mathbf{x}) = [2x_1, 4x_2]^\top$ 给出。我们将从初始位置 $[-5, -2]$ 通过梯度下降观察 $\mathbf{x}$ 的轨迹。

我们还需要两个辅助函数: 第一个是update函数, 并将其应用于初始值20次; 第二个函数会显示 $\mathbf{x}$ 的轨迹。

```
def train_2d(trainer, steps=20, f_grad=None):  #@save
    """用定制的训练机优化2D目标函数"""
    # s1和s2是稍后将使用的内部状态变量
    x1, x2, s1, s2 = -5, -2, 0, 0
    results = [(x1, x2)]
    for i in range(steps):
        if f_grad:
            x1, x2, s1, s2 = trainer(x1, x2, s1, s2, f_grad)
        else:
            x1, x2, s1, s2 = trainer(x1, x2, s1, s2)
        results.append((x1, x2))
    print(f'epoch {i + 1}, x1: {float(x1):f}, x2: {float(x2):f}')
    return results

def show_trace_2d(f, results):  #@save
    """显示优化过程中2D变量的轨迹"""
    d2l.set_figsize()
    d2l.plt.plot(*zip(*results), '-o', color='#ff7f0e')
    x1, x2 = torch.meshgrid(torch.arange(-5.5, 1.0, 0.1),
                             torch.arange(-3.0, 1.0, 0.1))
    d2l.plt.contour(x1, x2, f(x1, x2), colors='#1f77b4')
    d2l.plt.xlabel('x1')
    d2l.plt.ylabel('x2')
```

接下来, 我们观察学习率 $\eta = 0.1$ 时优化变量 $\mathbf{x}$ 的轨迹。可以看到, 经过20步之后, $\mathbf{x}$ 的值接近其位于 $[0, 0]$ 的最小值。虽然进展相当顺利, 但相当缓慢。


```
def f_2d(x1, x2): # 目标函数
    return x1 ** 2 + 2 * x2 ** 2

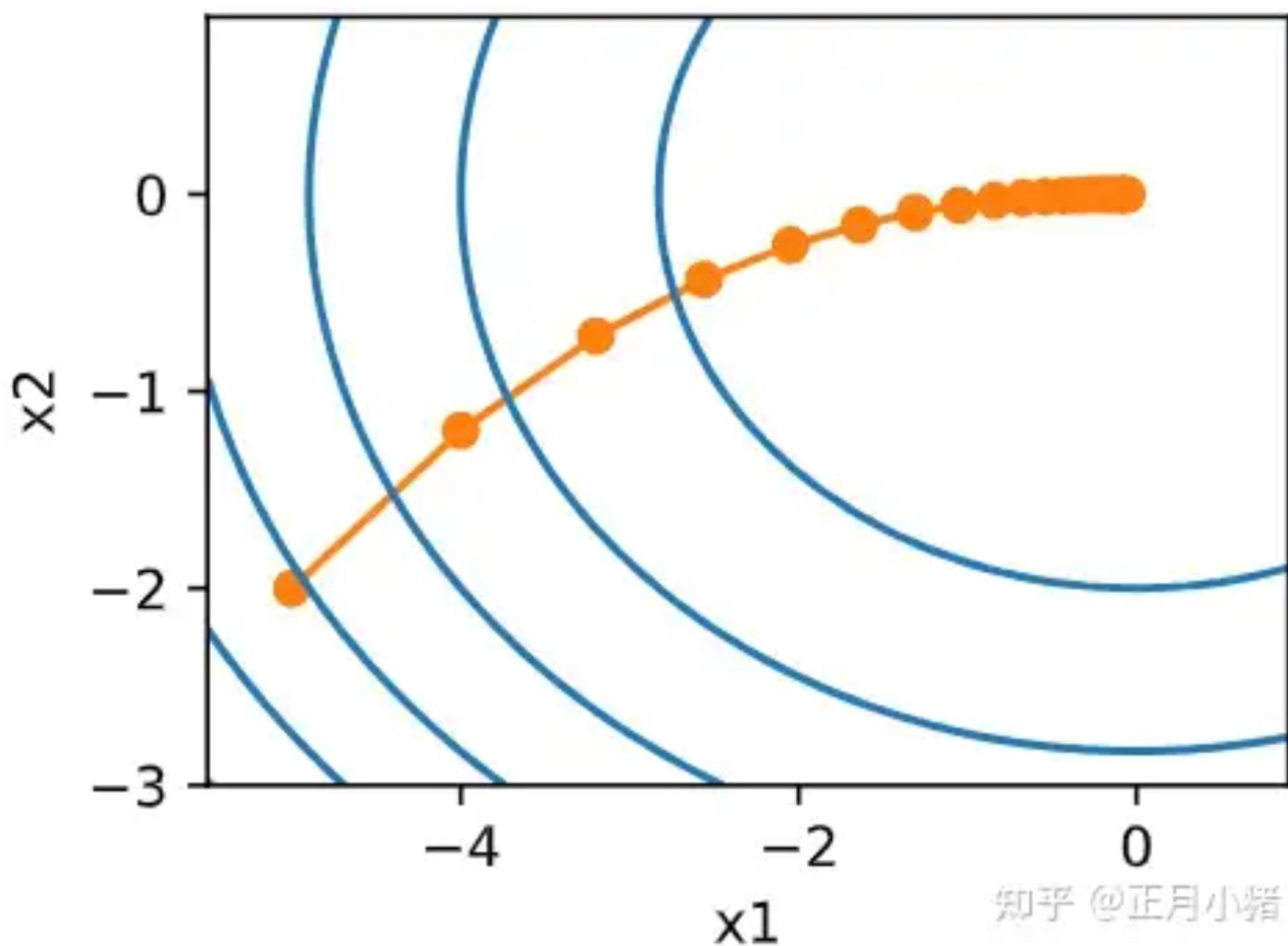
def f_2d_grad(x1, x2): # 目标函数的梯度
    return (2 * x1, 4 * x2)

def gd_2d(x1, x2, s1, s2, f_grad):
    g1, g2 = f_grad(x1, x2)
    return (x1 - eta * g1, x2 - eta * g2, 0, 0)

eta = 0.1
show_trace_2d(f_2d, train_2d(gd_2d, f_grad=f_2d_grad))
```

输出结果:

epoch 20, x1: -0.057646, x2: -0.000073



自适应方法

正如我们在 11.3.1.1 节中所看到的，选择“恰到好处”的学习率 η 是很棘手的。如果我们把它选得太小，就没有什么进展；如果太大，得到的解就会振荡，甚至可能发散。如果我们自动确定 η ，或者完全不必选择学习率，会怎么样？除了考虑目标函数的值和梯度、还考虑它的曲率的二阶方法可以帮助我们解决这个问题。虽然由于计算代价的原因，这些方法不能直接应用于深度学习，但它们为如何设计高级优化算法提供了有用的思维直觉，这些算法可以模拟下面概述的算法的许多理想特性。

牛顿法

回顾一些函数 $f : \mathbb{R}^d \rightarrow \mathbb{R}$ 的泰勒展开式，事实上我们可以把它写成

$$f(\mathbf{x} + \boldsymbol{\epsilon}) = f(\mathbf{x}) + \boldsymbol{\epsilon}^\top \nabla f(\mathbf{x}) + \frac{1}{2} \boldsymbol{\epsilon}^\top \nabla^2 f(\mathbf{x}) \boldsymbol{\epsilon} + \mathcal{O}(\|\boldsymbol{\epsilon}\|^3).$$
 gd-hot-taylor (11.3.8)

为了避免繁琐的符号，我们将 $\mathbf{H} \stackrel{\text{def}}{=} \nabla^2 f(\mathbf{x})$ 定义为 f 的 Hessian，是 $d \times d$ 矩阵。当 d 的值很小且问题很简单时， $\mathbf{H}$ 很容易计算。但是对于深度神经网络而言，考虑到 $\mathbf{H}$ 可能非常大， $\mathcal{O}(d^2)$ 个条目的存储代价会很高，此外通过反向传播进行计算可能雪上加霜。然而，我们姑且先忽略这些考量，看看会得到什么算法。

毕竟， f 的最小值满足 $\nabla f = 0$ 。遵循 2.4 节中的微积分规则，通过取 $\boldsymbol{\epsilon}$ 对 (11.3.8) 的导数，再忽略不重要的高阶项，我们便得到

$$\frac{\partial f(\mathbf{x} + \boldsymbol{\epsilon})}{\partial \boldsymbol{\epsilon}} = 0 + \nabla f(\mathbf{x}) + \frac{1}{2} \boldsymbol{\epsilon} (\nabla^2 f(\mathbf{x})^\top + \nabla^2 f(\mathbf{x})) + 0 = 0$$

$$\nabla f(\mathbf{x}) + \mathbf{H} \boldsymbol{\epsilon} = 0 \text{ and hence } \boldsymbol{\epsilon} = -\mathbf{H}^{-1} \nabla f(\mathbf{x}).$$
 (11.3.9)

也就是说，作为优化问题的一部分，我们需要将 Hessian 矩阵 $\mathbf{H}$ 求逆。

举一个简单的例子，对于 $f(x) = \frac{1}{2} x^2$ ，我们有 $\nabla f(x) = x$ 和 $\mathbf{H} = 1$ 。因此，对于任何 x ，我们可以获得 $\epsilon = -x$ 。换言之，单一步就足以完美地收敛，而无须任何调整。我们在这里比较幸运：泰勒展开式是确切的，因为 $f(x + \epsilon) = \frac{1}{2} x^2 + \epsilon x + \frac{1}{2} \epsilon^2$ 。

让我们看看其他问题。给定一个凸双曲余弦函数 c ，其中 c 为某些常数，我们可以看到经过几次迭代后，得到了 $x = 0$ 处的全局最小值。

```
c = torch.tensor(0.5)

def f(x):  # 0目标函数
    return torch.cosh(c * x)

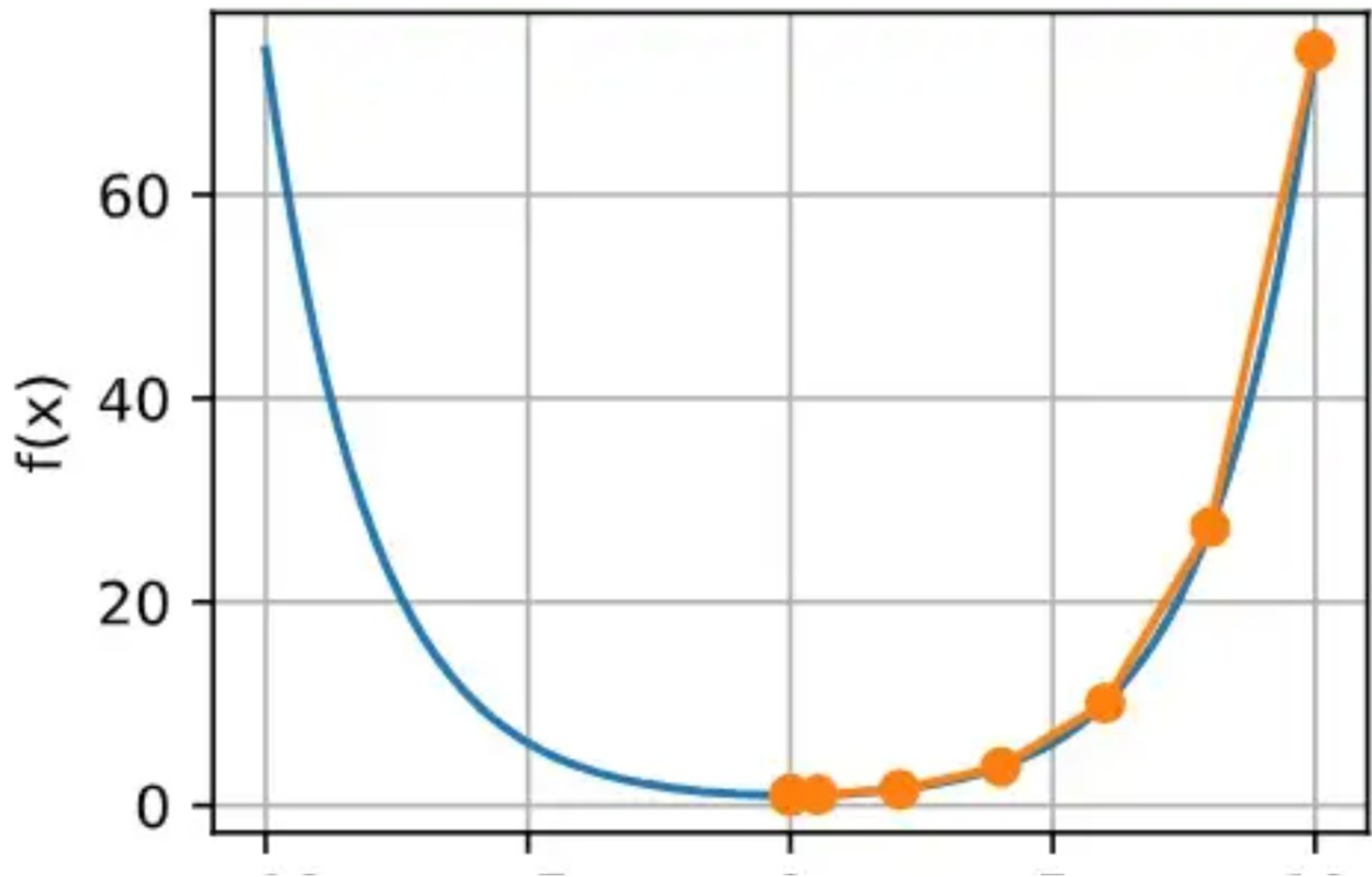
def f_grad(x):  # 目标函数的梯度
    return c * torch.sinh(c * x)

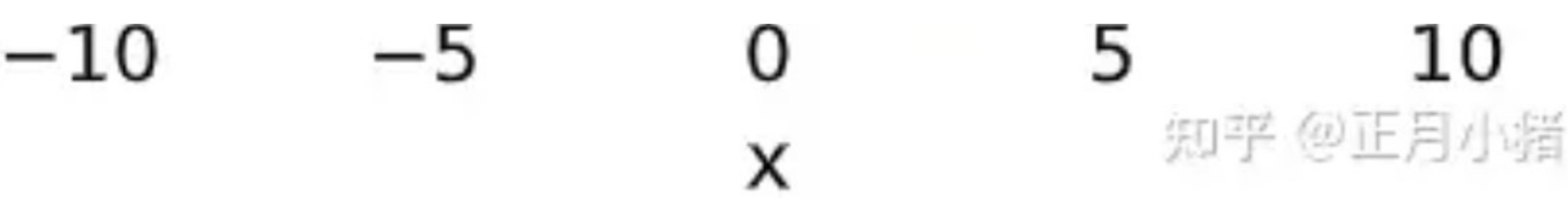
def f_hess(x):  # 目标函数的Hessian
    return c**2 * torch.cosh(c * x)

def newton(eta=1):
    x = 10.0
    results = [x]
    for i in range(10):
        x -= eta * f_grad(x) / f_hess(x)
        results.append(float(x))
    print('epoch 10, x:', x)
    return results

show_trace(newton(), f)
```

输出结果：
epoch 10, x: tensor(0.)





现在让我们考虑一个非凸函数，比如 $f(x) = x \cos(cx)$ ， c 为某些常数。请注意在牛顿法中，我们最终将除以Hessian。这意味着如果二阶导数是负的， f 的值可能会趋于增加。这是这个算法的致命缺陷！让我们看看实践中会发生什么。

```
c = torch.tensor(0.15 * np.pi)

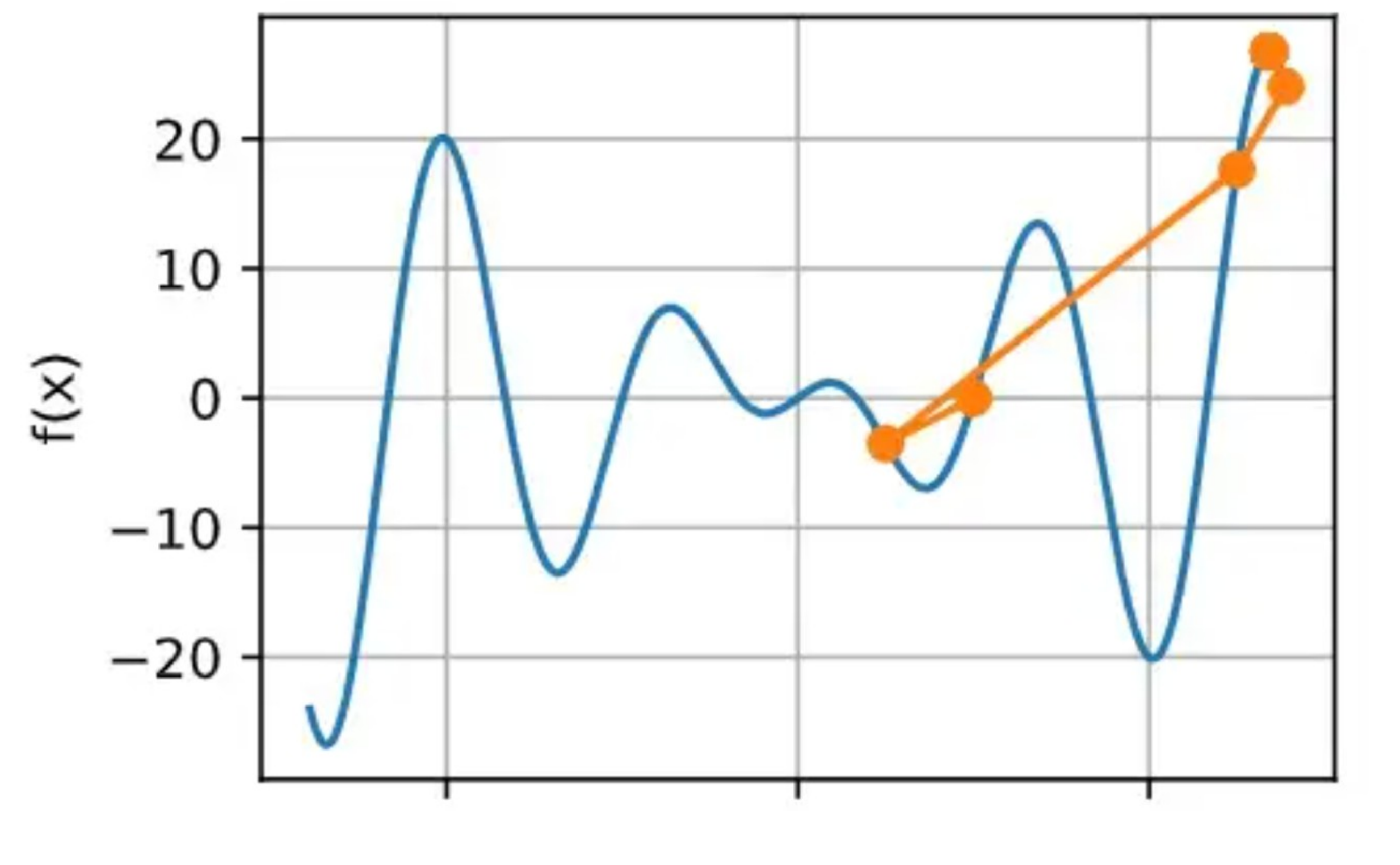
def f(x): # 目标函数
    return x * torch.cos(c * x)

def f_grad(x): # 目标函数的梯度
    return torch.cos(c * x) - c * x * torch.sin(c * x)

def f_hess(x): # 目标函数的Hessian
    return - 2 * c * torch.sin(c * x) - x * c**2 * torch.cos(c * x)

show_trace(newton(), f)
```

输出结果：
epoch 10, x: tensor(26.8341)



-20020

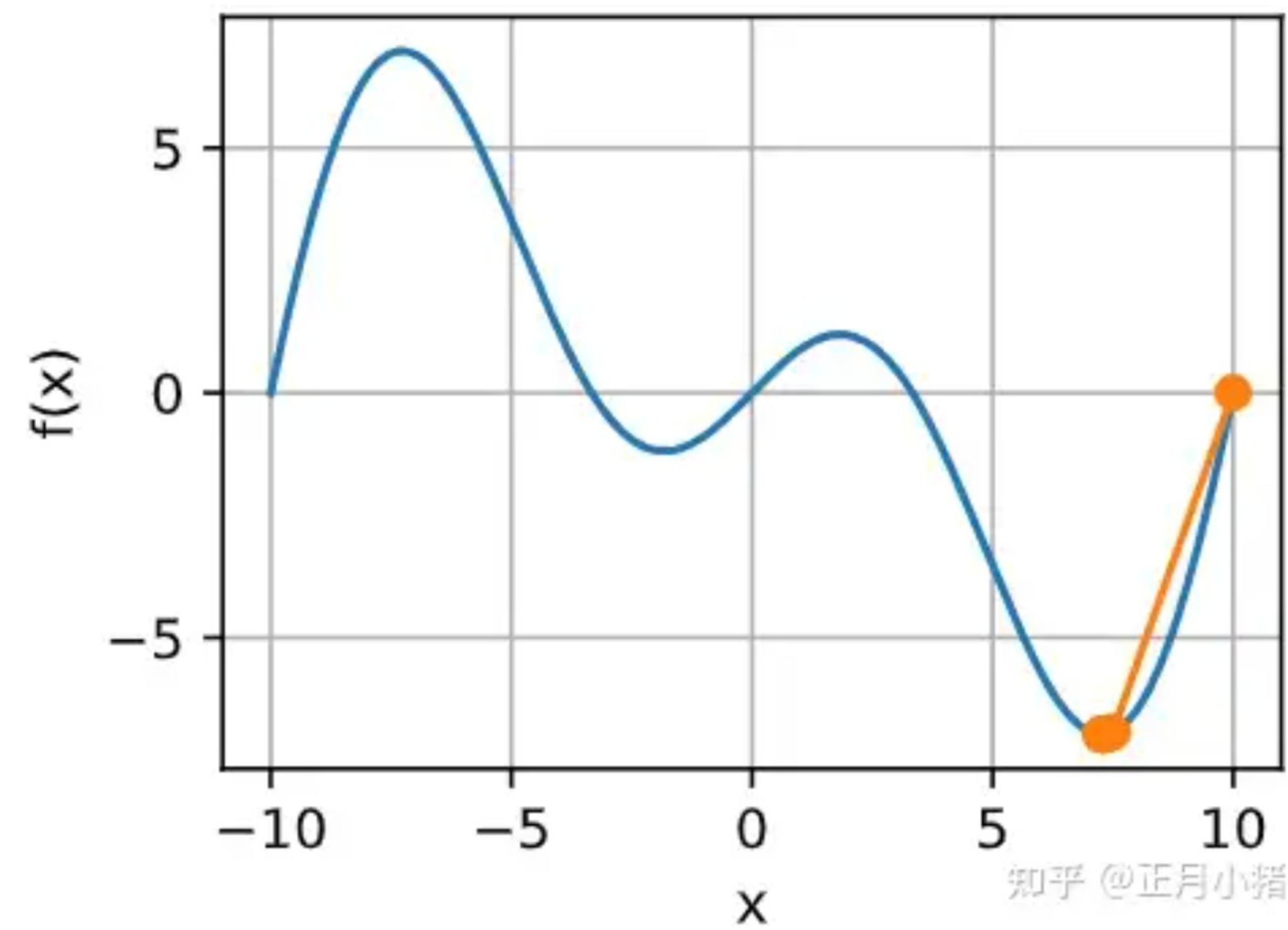
x

知乎 @正月小嘴

这发生了惊人的错误。我们怎样才能修正它？一种方法是用取Hessian的绝对值来修正，另一个策略是重新引入学习率。这似乎违背了初衷，但不完全是——拥有二阶信息可以使我们在曲率较大时保持谨慎，而在目标函数较平坦时则采用较大的学习率。让我们看看在学习率稍小的情况下它是如何生效的，比如 $\eta = 0.5$ 。如我们所见，我们有了一个相当高效的算法。

```
show_trace(newton(0.5), f)
```

输出结果：
epoch 10, x: tensor(7.2699)



收敛性分析

在此，我们以部分目标凸函数 f 为例，分析它们的牛顿法收敛速度。这些目标凸函数三次可微，而且二阶导数不为零，即 $f'' > 0$ 。由于多变量情况下的证明是对以下一维参数情况证明的直接拓展，对我们理解这个问题不能提供更多帮助，因此我们省略了多变量情况的证明。

用 $x^{(k)}$ 表示 x 在第 k^{th} 次迭代时的值, 令 $e^{(k)} \stackrel{\text{def}}{=} x^{(k)} - x^*$ 表示 k^{th} 迭代时与最优性的距离。通过泰勒展开, 我们得到条件 $f'(x) = 0$ 可以写成

$$0 = f'(x^{(k)} - e^{(k)}) = f'(x^{(k)}) - e^{(k)} f''(x^{(k)}) + \frac{1}{2}(e^{(k)})^2 f'''(\xi^{(k)}), \tag{11.3.10}$$

这对某些 $\xi^{(k)} \in [x^{(k)} - e^{(k)}, x^{(k)}]$ 成立。将上述展开除以 $f''(x^{(k)})$ 得到

$$e^{(k)} - \frac{f'(x^{(k)})}{f''(x^{(k)})} = \frac{1}{2}(e^{(k)})^2 \frac{f'''(\xi^{(k)})}{f''(x^{(k)})}. \tag{11.3.11}$$

回想之前的方程 $x^{(k+1)} = x^{(k)} - f'(x^{(k)})/f''(x^{(k)})$ 。代入这个更新方程, 取两边的绝对值, 我们得到

$$|e^{(k+1)}| = \frac{1}{2}(e^{(k)})^2 \frac{|f'''(\xi^{(k)})|}{f''(x^{(k)})}. \tag{11.3.12}$$

因此, 每当我们处于有界区域 $|f'''(\xi^{(k)})|/(2f''(x^{(k)})) \leq c$, 我们就有一个二次递减误差

$$|e^{(k+1)}| \leq c(e^{(k)})^2.$$

另一方面, 优化研究人员称之为“线性”收敛, 而将 $|e^{(k+1)}| \leq \alpha |e^{(k)}|$ 这样的条件称为“恒定”收敛速度。请注意, 我们无法估计整体收敛的速度, 但是一旦我们接近极小值, 收敛将变得非常快。另外, 这种分析要求 f 在高阶导数上表现良好, 即确保 f 在如何变化它的值方面没有任何“超常”的特性。

预处理

计算和存储完整的Hessian非常昂贵, 而改善这个问题的一种方法是“预处理”。它回避了计算整个Hessian, 而只计算“对角线”项, 即如下的算法更新:

$$\mathbf{x} \leftarrow \mathbf{x} - \eta \text{diag}(\mathbf{H})^{-1} \nabla f(\mathbf{x}).$$

虽然这不如完整的牛顿法精确, 但它仍然比不使用要好得多。为什么预处理有效呢? 假设一个变量以毫米表示高度, 另一个变量以公里表示高度的情况。假设这两种自然尺度都以米为单位, 那么我们的参数化就出现了严重的不匹配。幸运的是, 使用预处理可以消除这种情况。梯度下降的有效预处理相当于为每个变量选择不同的学习率(矢量 $\mathbf{x}$ 的坐标)。我们将在后面一节看到, 预处理推动了随机梯度下降优化算法的一些创新。

梯度下降和线搜索

梯度下降的一个关键问题是我们可能会超过目标或进展不足，解决这一问题的简单方法是结合使用线搜索和梯度下降。也就是说，我们使用 $\nabla f(\mathbf{x})$ 给出的方向，然后进行二分搜索，以确定哪个学习率 η 使 $f(\mathbf{x} - \eta \nabla f(\mathbf{x}))$ 取最小值。

有关分析和证明，此算法收敛迅速（请参见 [\(Boyd and Vandenberghe, 2004\)](#)）。然而，对深度学习而言，这不太可行。因为线搜索的每一步都需要评估整个数据集上的目标函数，实现它的方式太昂贵了。

小结

- 学习率的大小很重要：学习率太大会使模型发散，学习率太小会没有进展。
- 梯度下降会可能陷入局部极小值，而得不到全局最小值。
- 在高维模型中，调整学习率是很复杂的。
- 预处理有助于调节比例。
- 牛顿法在凸问题中一旦开始正常工作，速度就会快得多。
- 对于非凸问题，不要不作任何调整就使用牛顿法。

练习

1. 用不同的学习率和目标函数进行梯度下降实验。

解：

对于11.3.1.2中的例子，降低学习率即可快速找到一个很好的局部最小值。

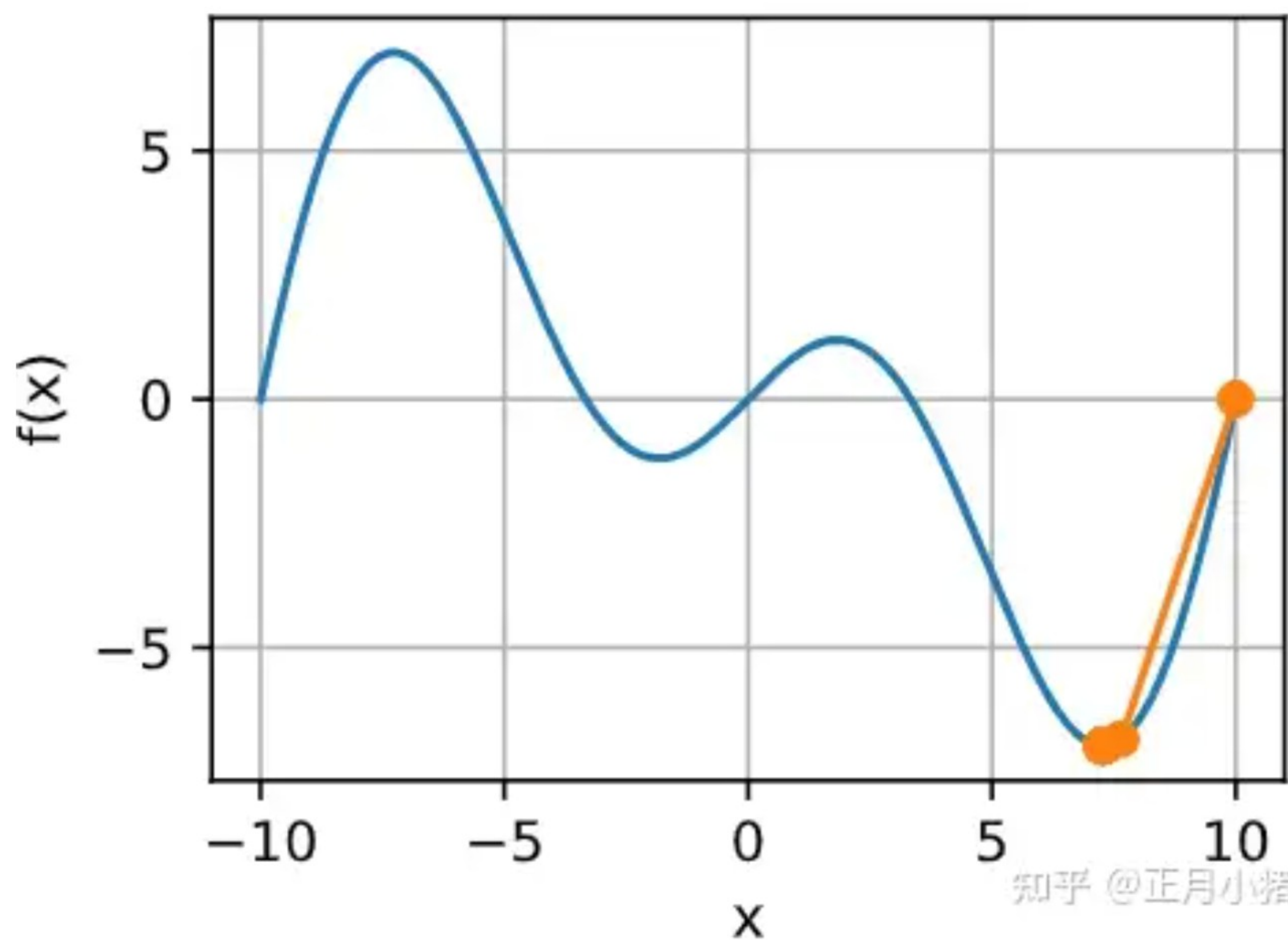
代码如下：

```
c = torch.tensor(0.15 * np.pi)

def f(x): # 目标函数
    return x * torch.cos(c * x)

def f_grad(x): # 目标函数的梯度
    return torch.cos(c * x) - c * x * torch.sin(c * x)

show_trace(gd(0.5, f_grad), f)
epoch 10, x: 7.269388
```

2. 在区间 $[a, b]$ 中实现线搜索以最小化凸函数。

- 1) 是否需要导数来进行二分搜索，即决定选择 $[a, (a + b)/2]$ 还是 $[(a + b)/2, b]$ 。
- 2) 算法的收敛速度有多快？
- 3) 实现该算法，并将其应用于求 $\log(\exp(x) + \exp(-2x - 3))$ 的最小值。

解：

- 1) 二分搜索（或称为二分法）是一种通过不断地将搜索区间对半分割来逼近最小值的方法。每次迭代中，算法选择当前区间的中点 $(a + b)/2$ ，并根据目标函数在这个点的值与端点处的值比较，决定下一步搜索哪个子区间。因此，二分搜索主要依赖于函数值而非导数值，但要求函数是单峰的（即在搜索区间内只有一个局部极小值）。
- 2) 二分搜索的收敛速度是线性的。每一步迭代后，搜索区间的长度减半，故算法的n次迭代的收敛速度是 $\mathcal{O}(\log n)$ 。
- 3) 对于 $\log(\exp(x) + \exp(-2x - 3))$ ，其一阶导数为：

$$\frac{\exp(x)-2\exp(-2x-3)}{\exp(x)+\exp(-2x-3)}$$

一阶导数的分母 $\exp(x) + \exp(-2x - 3)$ 恒正，因此一阶导数的正负性由分子部分决定。

考虑分子 $\exp(x) - 2\exp(-2x - 3)$ ：

- 当 x 趋向于正无穷时， $\exp(x)$ 趋向于正无穷，而 $\exp(-2x - 3)$ 趋向于 0，因此分子趋向于正无穷。
- 当 x 趋向于负无穷时， $\exp(x)$ 趋向于 0，而 $\exp(-2x - 3)$ 趋向于正无穷，因此分子趋向于负无穷。令 $\exp(x) - 2\exp(-2x - 3) = 0$ ，可解得最小值 $x_0 = \frac{\ln(2)-3}{3}$ 。因此，一阶导数在 $x < \frac{\ln(2)-3}{3}$ 时，函数单调减；在 $x > \frac{\ln(2)-3}{3}$ 时，函数单调增。

代码如下：

```
import math

def f(x):
    return math.log(math.exp(x) + math.exp(-2*x - 3))

def f_grad(x):
    return (math.exp(x) - 2*math.exp(-2*x - 3))/(math.exp(x) + math.exp(-2*x - 3))

# 二分搜索, d为distance, a和b之间的最小距离
def bisection(a, b, f, d):

    results = [a,b]
    while ((b - a) >= d):
        c = (a + b) / 2
        if f(c) < f(b):
            b = c
        else:
            a = c
        results.append(float(c))
    print(f"函数的最小值点为 x = {c:.2f}, 最小值为 f(x) = {f(c):.2f}")
    return results

a = -10
b = 10
d = 1e-6

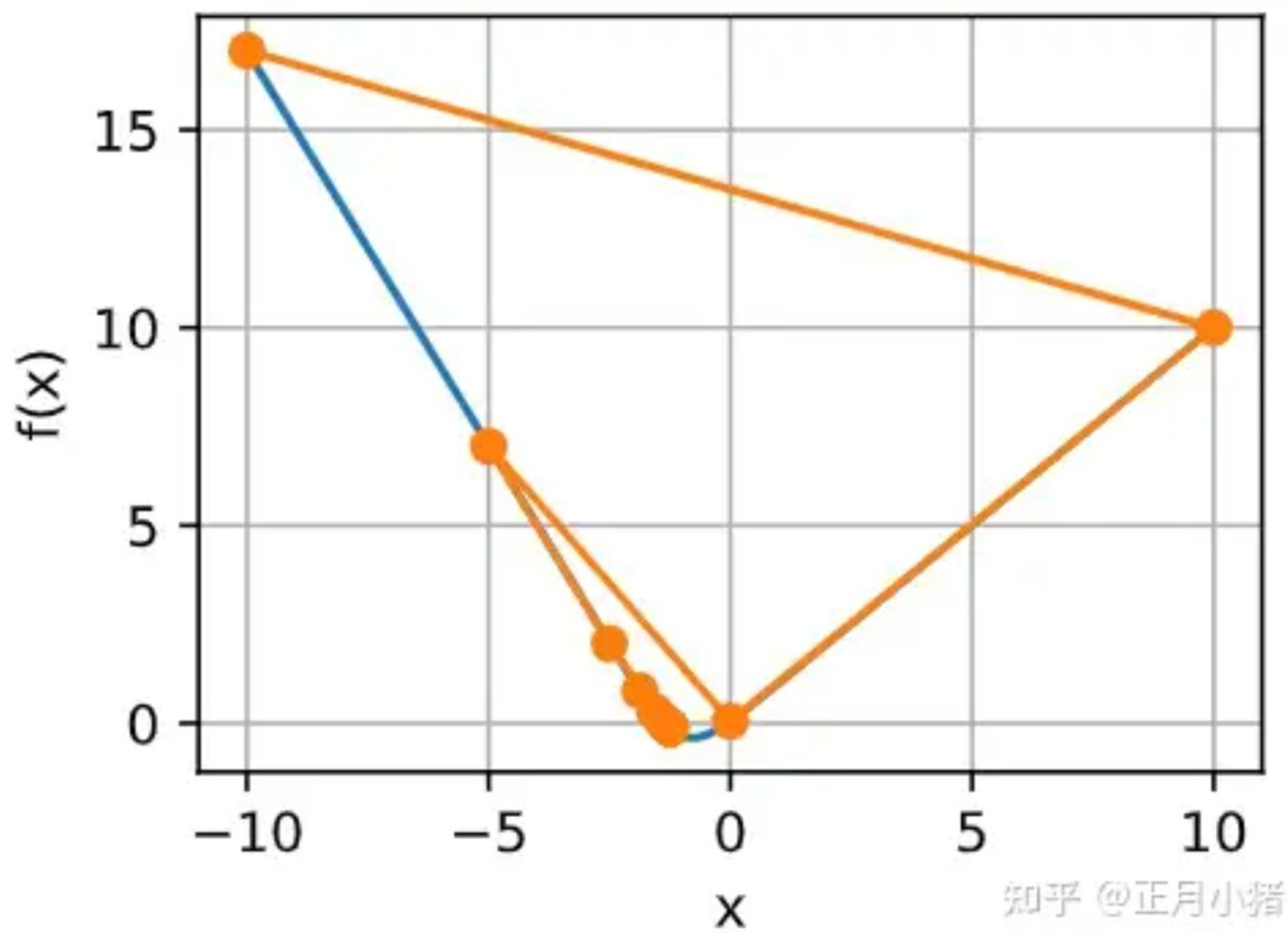
results = bisection(a, b, f, d)
```

输出结果：

函数的最小值点为 $x = -1.25$, 最小值为 $f(x) = -0.11$


```
show_trace(results, f)
```

输出结果：



3. 设计一个定义在 $\mathbb{R}^2$ 上的目标函数，它的梯度下降非常缓慢。提示：不同坐标的缩放方式不同。

解：
可以设计一个椭圆函数 $f(x, y) = \frac{x^2}{a^2} + \frac{y^2}{b^2}$ ，其中 a 和 b 是椭圆的半轴长度。如果 a 远大于 b ，那么函数在 x 轴方向上的变化就会非常缓慢，而在 y 轴方向上的变化就会相对快速。
设 $a = 100, b = 1$ ，目标函数为：

$$f(x, y) = \frac{x^2}{10000} + y^2$$

图像如下：

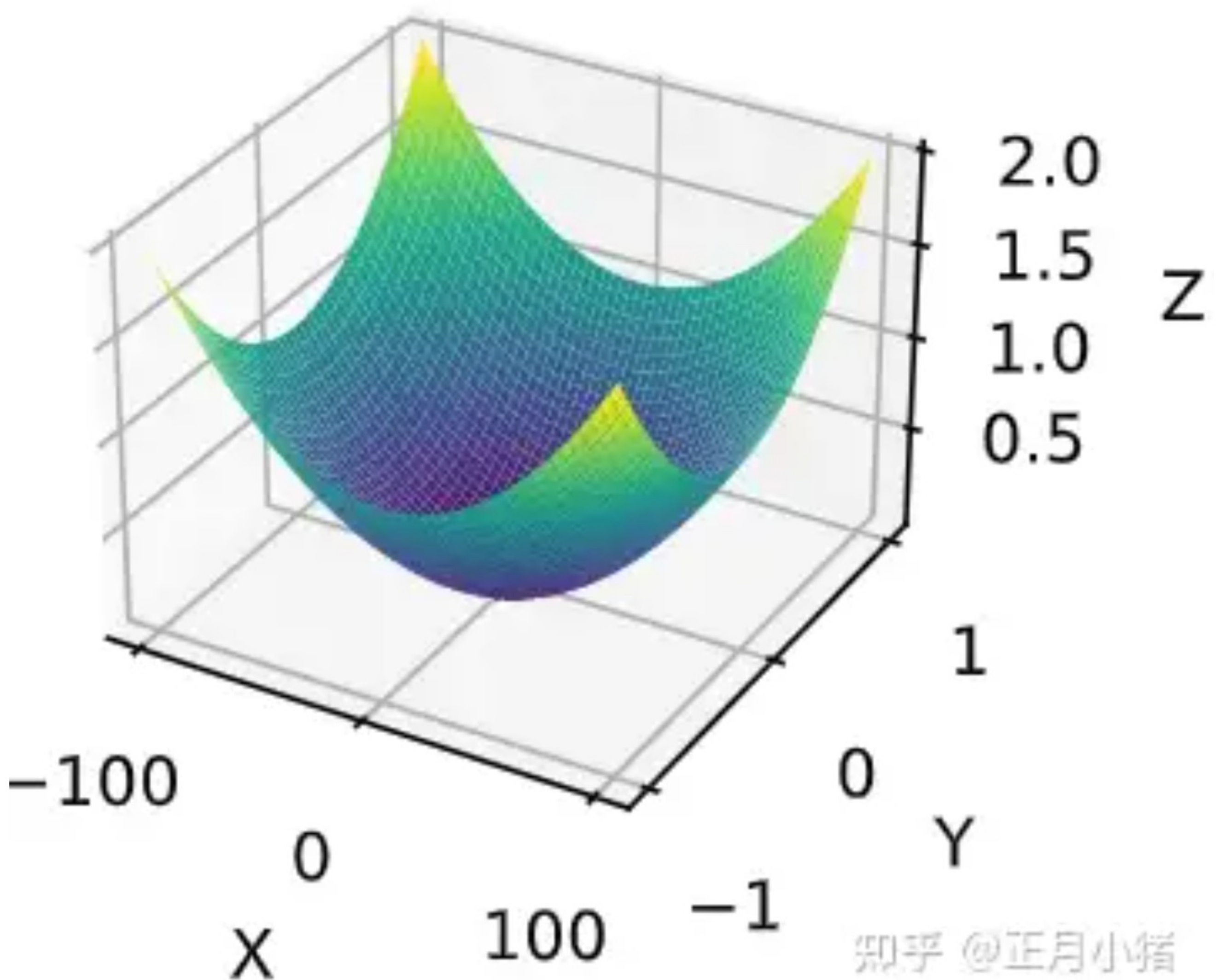
```
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

x = np.linspace(-100, 100, 400)
y = np.linspace(-1, 1, 400)

X, Y = np.meshgrid(x, y)
Z = X**2 / 10000 + Y**2

fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.plot_surface(X, Y, Z, cmap='viridis')
ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('Z')

plt.show()
```



知乎 @正月小猪

4. 使用预处理实现牛顿方法的轻量版本。

- 1) 使用对角Hessian作为预条件子。
- 2) 使用它的绝对值，而不是实际值（可能有符号）。
- 3) 将此应用于上述问题。

解：

- 1) 使用对角Hessian作为预条件子： $\mathbf{D} = \text{diag}(H)$
- 2) 使用对角Hessian的绝对值： $\mathbf{D}_{\text{abs}} = \text{diag}(|H|)$
- 3) 应用于上一题的椭圆目标函数中： 对于函数 $f(x, y) = \frac{x^2}{10000} + y^2$,

$$H = \begin{bmatrix} \frac{2}{10000} & 0 \\ 0 & 2 \end{bmatrix}$$

$$\mathbf{D} = \begin{bmatrix} \frac{2}{10000} & 0 \\ 0 & 2 \end{bmatrix}$$

$$\mathbf{D}_{\text{abs}} = \begin{bmatrix} \frac{2}{10000} & 0 \\ 0 & 2 \end{bmatrix}$$

在每次迭代中，更新规则如下：

$$\mathbf{x} \leftarrow \mathbf{x} - \eta \mathbf{D}_{\text{abs}}^{-1} \nabla f(\mathbf{x})$$

其中 η 是学习率， $\nabla f(\mathbf{x})$ 是函数的梯度， $\mathbf{D}_{\text{abs}}^{-1} = \begin{bmatrix} \frac{10000}{2} & 0 \\ 0 & \frac{1}{2} \end{bmatrix}$.

代码实现如下：

```
def f(x):  
    return x[0]**2 / 10000 + x[1]**2  
  
def f_grad(x):  
    return np.array([x[0] / 5000, 2 * x[1]])  
  
def hessian_diag(x):  
    return np.array([1/5000, 2])  
  
def light_newton_method(x, lr, epoch):  
    x = x  
    results = [x]  
    for i in range(epoch):  
        # 计算梯度  
        gradient = f_grad(x)  
        # 计算对角Hessian的绝对值  
        hessian_diag_values = abs(hessian_diag(x))  
        # 计算预条件子的逆  
        D_inv = np.diag(1 / np.abs(hessian_diag_values))  
        x = x - lr * np.dot(D_inv, gradient)  
        results.append(x)  
    print(f"epoch {i+1}: x = {x[0]:2f}, y = {x[1]:2f}, f(x) = {f(x):2f}")  
    return results  
  
x = np.array([10, 10])  
lr, epoch = 0.5, 10  
  
results = light_newton_method(x, lr, epoch)
```

输出结果:

epoch 10: x = 0.009766, y = 0.009766, f(x) = 0.000095

5. 将上述算法应用于多个目标函数（凸或非凸）。如果把坐标旋转45度会怎么样?

解:

1) 将轻量版的牛顿方法应用于凸函数或非凸函数，其核心思想是相同的：使用对角Hessian作为预条件子，并取其绝对值来更新参数。对于不同的函数，只需要计算每个函数的梯度和Hessian矩阵的对角元素，然后按照相同的更新规则进行迭代。

2) 当坐标系旋转45度时，可以通过一个旋转矩阵来实现：

$$R = \begin{bmatrix} \cos(45^\circ) & -\sin(45^\circ) \\ \sin(45^\circ) & \cos(45^\circ) \end{bmatrix} = \begin{bmatrix} \frac{\sqrt{2}}{2} & -\frac{\sqrt{2}}{2} \\ \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \end{bmatrix}$$

点 (x, y) 旋转后的新坐标 (x', y') 为:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = R \begin{bmatrix} x \\ y \end{bmatrix}$$

即:

$$x' = \frac{\sqrt{2}}{2}x - \frac{\sqrt{2}}{2}y$$

$$y' = \frac{\sqrt{2}}{2}x + \frac{\sqrt{2}}{2}y$$

代码如下:

```
# 旋转坐标矩阵
```

```
def rotation_matrix(angle):  
    rad = np.deg2rad(angle)  
    return np.array([[np.cos(rad), -np.sin(rad)], [np.sin(rad), np.cos(rad)]])
```

```
def light_newton_method_with_rotation_matrix(x, lr, epoch, angle):  
    x = x  
    results = [x.copy()]  
    R = rotation_matrix(angle)  
    for i in range(epoch):  
        # 旋转坐标  
        x_rotated = np.dot(R, x)  
  
        gradient = f_grad(x_rotated)  
        hessian_diag_values = abs(hessian_diag(x_rotated))  
        D_inv = np.diag(1 / np.abs(hessian_diag_values))  
        gradient_rotated_back = np.dot(R.T, np.dot(D_inv, gradient))  
        x = x - lr * gradient_rotated_back  
        results.append(x.copy())  
    print(f"epoch {i+1}: x = {x[0]:2f}, y = {x[1]:2f}, f(x) = {f(x):2f}")  
    return np.array(results)
```

```
x = np.array([10, 10])
```

```
lr, epoch, angle = 0.5, 10, 45
```

```
results = light_newton_method_with_rotation_matrix(x, lr, epoch, angle)
```

```
x = np.linspace(-100, 100, 400)
y = np.linspace(-10, 10, 400)

X, Y = np.meshgrid(x, y)
Z = X**2 / 10000 + Y**2

fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.plot_surface(X, Y, Z, cmap='viridis', alpha=0.5)

# 绘制梯度下降过程
ax.plot(np.array(results[:, 0]), np.array(results[:, 1]), f(np.array([results[:, 0], results[:, 1]])))

ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('Z')

plt.show()
```

